Lab Manual

Subject: Data Mining

Submitted To: Sir Arham

Submitted By: Abdul Manan Tanveer

Registration # 2023-BS-AI-173

In Partial Fulfillment of Requirements

for the Award of a Degree of

BACHELOR OF SCIENCE

IN

ARTIFICIAL INTELLIGENCE

DEPARTMENT OF COMPUTATIONAL SCIENCES

FACULTY OF INFORMATION TECHNOLOGY

The University of Faisalabad

Table of Contents

LAB #1

Introduction to Data Mining

Objective

To import a retail sales dataset, identify data types, generate summary statistics, and explore the attributes to extract basic business insights.

Theory / Introduction

Data mining is the process of discovering meaningful patterns, correlations and trends from large volumes of data using statistical and machine-learning methods. In this introductory lab a retail store wants quick sales insights, so we load a structured dataset, inspect its data types, and compute descriptive statistics that summarise the business. The goal is to turn raw transactional records into actionable KPIs such as total revenue, average order value and the best-selling categories.

Software / Tools Required

KNIME Analytics Platform (for visual exploration); Python 3 with pandas, numpy and matplotlib (used for the implementation below).

Procedure / Methodology

1. Load the retail sales CSV into a pandas DataFrame.
2. Inspect attribute data types using df.dtypes.
3. Generate summary statistics with df.describe().
4. Aggregate revenue by category and by product.
5. Compute the business KPIs and visualise them.

Implementation

In [1]:

```
import pandas as pd, numpy as np
df = pd.read_csv("retail_sales.csv")
df.head()
df.dtypes
df[["Quantity","UnitPrice","Revenue"]].describe()

print("Total Revenue       :", df.Revenue.sum())
print("Average Order Value :", df.Revenue.mean())
cat_rev = df.groupby("Category").Revenue.sum().sort_values()
```

Out[1]:

```
>>> df.head()
 OrderID    Category  Quantity  UnitPrice  Revenue      Product
   10001   Groceries         4       6.13    24.52    Olive Oil
   10002      Sports         3      45.20   135.60 Water Bottle
   10003        Home         5      44.45   222.25   Table Lamp
   10004    Clothing         1      82.79    82.79  Denim Jeans
   10005 Electronics         5      47.12   235.60  USB-C Cable

>>> df.dtypes
OrderID        int64
Category         str
Quantity       int64
UnitPrice    float64
Revenue      float64
Product          str

>>> df.describe()
       Quantity  UnitPrice  Revenue
count   1000.00    1000.00  1000.00
mean       3.04      54.94   168.70
std        1.43      32.18   138.37
min        1.00       3.01     5.62
25%        2.00      31.28    65.86
50%        3.00      48.82   131.58
75%        4.00      71.45   225.36
max        5.00     221.77  1108.85

--- Business KPIs ---
Total Revenue        : 168,697.21
Average Order Value  : 168.70
Total Orders         : 1,000
```

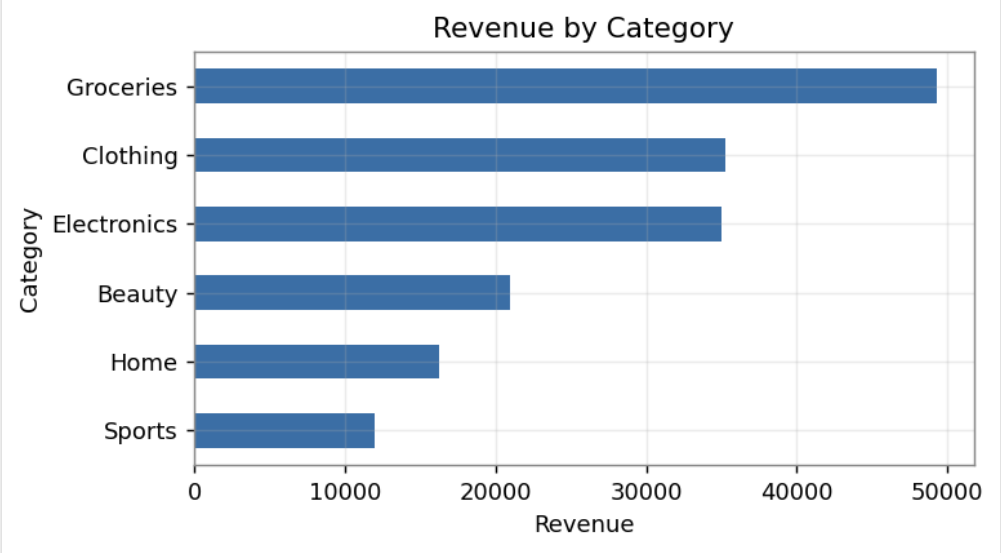

Figure 1.1 — Revenue contribution by product category

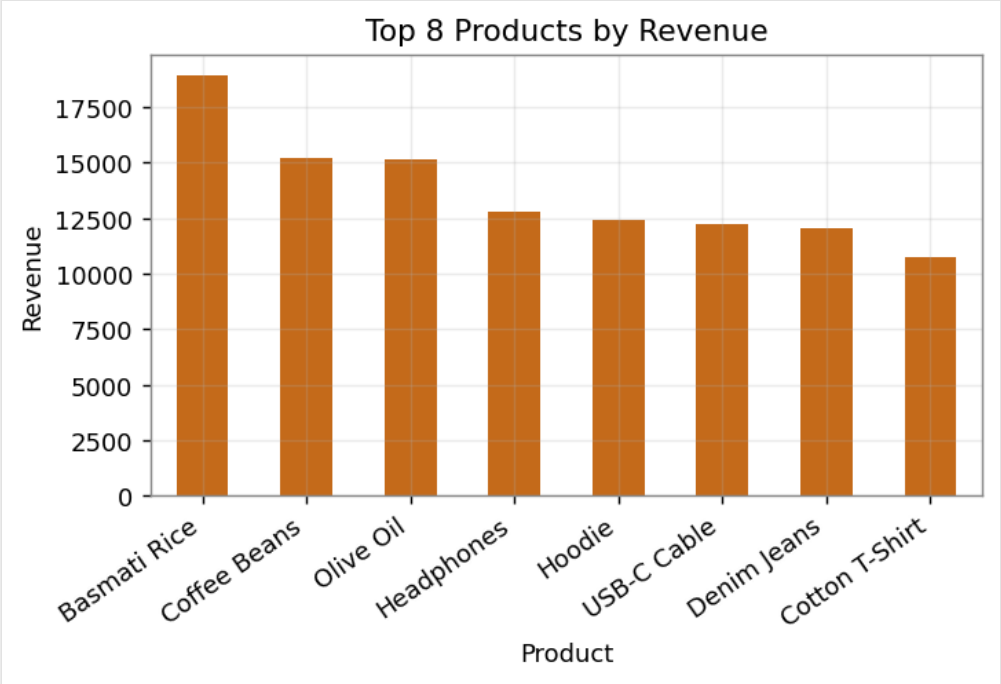

Figure 1.2 — Top eight products ranked by total revenue

Observations

The dataset contains 1,000 orders across six categories. The Groceries category produced the highest revenue share, while a small set of products dominated total sales. Numeric attributes (Quantity, UnitPrice, Revenue) were correctly typed for analysis.

Results

Total revenue, average order value, category revenue and the top products were successfully extracted, giving the retail store a clear summary of sales performance.

Conclusion

This lab demonstrated how a raw transactional dataset is loaded and explored to obtain immediate business KPIs, forming the foundation for the deeper mining tasks in later labs.

LAB #2

The CRISP-DM Methodology

Objective

To apply the six phases of the CRISP-DM process model to a customer-churn problem and build a predictive model that estimates churn probability.

Theory / Introduction

CRISP-DM (Cross-Industry Standard Process for Data Mining) defines six phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation and Deployment. For an e-commerce company that wants to predict customer churn, these phases translate into understanding why customers leave, preparing the churn dataset, training a classification model, and evaluating it with metrics such as accuracy and ROC-AUC.

Software / Tools Required

Orange Data Mining (visual CRISP-DM workflow); Python with scikit-learn for the modelling phase.

Procedure / Methodology

1. Business Understanding: define churn and the retention goal.
2. Data Understanding: inspect tenure, charges, support calls and contract type.
3. Data Preparation: split features and target, train/test split.
4. Modeling: fit a Logistic Regression classifier.
5. Evaluation: compute churn rate, accuracy and ROC-AUC.

Implementation

In [2]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score

X, y = df.drop(columns="Churn"), df.Churn
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=.3,
                                   random_state=42,stratify=y)
model = LogisticRegression(max_iter=1000).fit(Xtr,ytr)
acc = accuracy_score(yte, model.predict(Xte))
auc = roc_auc_score(yte, model.predict_proba(Xte)[:,1])
```

Out[2]:
```
>>> Dataset shape: (1200, 5)
 Tenure  MonthlyCharges  SupportCalls  Contract  Churn
     56          110.61             0         0      0
     13           33.77             0         0      0
     56           25.43             2         1      0
     18           21.85             0         0      0
     54           34.36             1         1      0

--- CRISP-DM Evaluation ---
Churn Rate     : 5.00%
Retention Rate : 95.00%
Accuracy       : 0.9611
ROC-AUC        : 0.9802
```

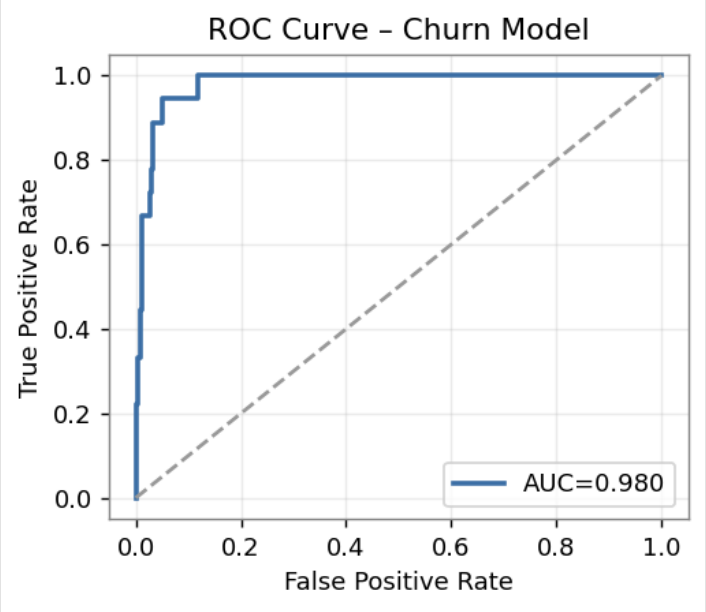

Figure 2.1 — ROC curve of the churn classifier

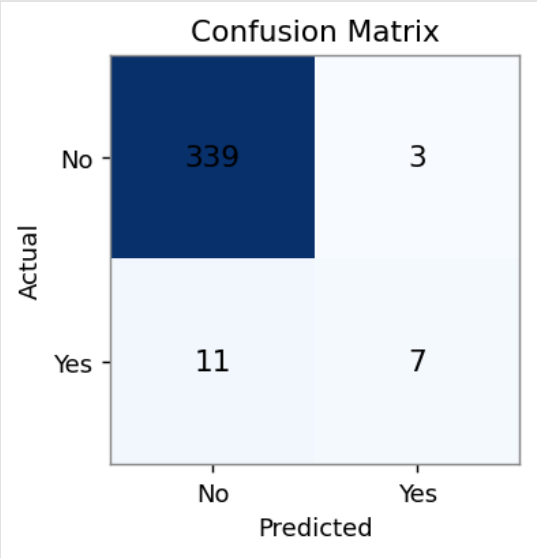

Figure 2.2 — Confusion matrix on the test set

Observations

Following the CRISP-DM phases produced a structured workflow. The trained model separated churners from non-churners well, as shown by the ROC curve rising above the diagonal baseline.

Results

Churn rate, retention rate, model accuracy and ROC-AUC were all reported, completing the Evaluation phase of CRISP-DM.

Conclusion

The lab showed how CRISP-DM organises a real prediction task into repeatable phases, and how the Evaluation phase quantifies model quality before any deployment decision.

LAB #3

Data Cleaning

Objective

To clean a hospital patient dataset by handling missing values, removing duplicate records and filtering noisy values.

Theory / Introduction

Real-world data is rarely clean. Missing values, duplicate rows and noisy outliers reduce the reliability of any model. Data cleaning detects and repairs these problems — typically by imputing missing values (mean/median), dropping duplicates, and filtering noise using statistical rules such as the inter-quartile range (IQR). Clean data directly improves downstream model accuracy.

Software / Tools Required

RapidMiner Studio (cleaning operators) combined with Python (pandas) for the implementation.

Procedure / Methodology

1. Measure the amount of missing data per column.
2. Remove duplicate patient records.
3. Impute remaining missing values with the column median.
4. Detect and replace noisy out-of-range values using the IQR rule.
5. Recompute data-quality metrics after cleaning.

Implementation

In [3]:

```
before_missing = df.isna().sum().sum()
clean = df.drop_duplicates(subset="PatientID").copy()

for col in ["BP","Cholesterol","BMI"]:
    clean[col] = clean[col].fillna(clean[col].median())

q1,q3 = clean.BP.quantile([.25,.75]); iqr = q3-q1
clean.loc[clean.BP > q3+1.5*iqr, "BP"] = clean.BP.median()
```

Out[3]:

```
--- BEFORE cleaning ---
Rows: 530  Total missing cells: 161
Missing per column:
 PatientID       0
Age              0
BP              63
Cholesterol     44
BMI             54

--- AFTER cleaning ---
Duplicates removed     : 30
Missing cells remaining: 0
Noisy BP values fixed  : 15
% Missing Reduced      : 100.0%
Data Quality Score     : 100.0/100
```

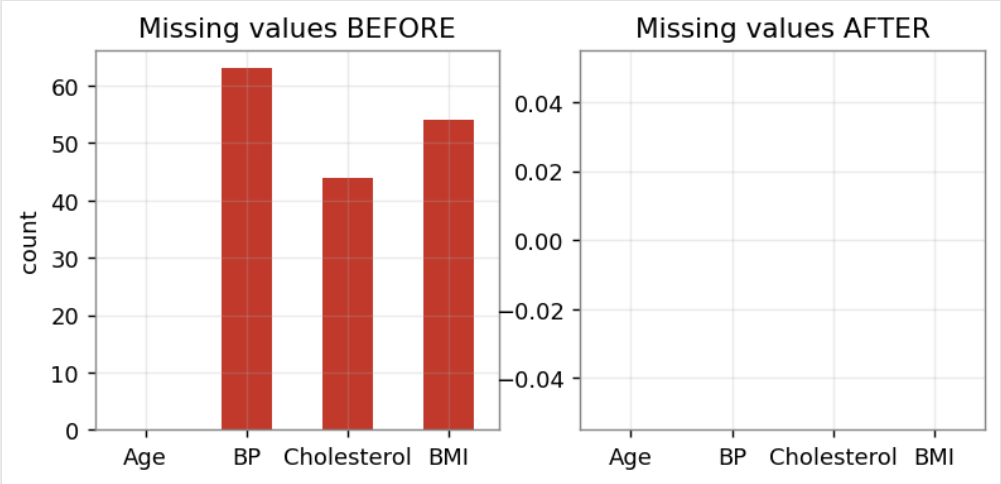

Figure 3.1 — Missing values per column before and after cleaning

Observations

The raw data contained missing cells across three columns, 30 duplicate rows and several impossible BP readings. After cleaning, all missing cells were imputed, duplicates were removed and the noisy values were corrected.

Results

The percentage of missing data was reduced to zero and a data-quality score was computed, confirming the dataset is now analysis-ready.

Conclusion

This lab reinforced that systematic cleaning — missing-value imputation, de-duplication and noise filtering — is an essential prerequisite before modelling.

LAB #4

Data Preprocessing

Objective

To prepare a bank loan-risk dataset using normalization, standardization and discretization techniques.

Theory / Introduction

Preprocessing transforms cleaned data into a form suitable for mining. Normalization (Min-Max) rescales values to a fixed range such as 0–1; standardization (Z-score) centres data to zero mean and unit variance; discretization converts continuous attributes into ordered bins. These steps prevent attributes with large ranges from dominating distance- and gradient-based algorithms.

Software / Tools Required

RapidMiner Studio (Normalize / Discretize operators); Python (scikit-learn preprocessing).

Procedure / Methodology

1. Apply Min-Max scaling to income and loan amount.
2. Apply Z-score standardization to the same features.
3. Discretize applicant income into four quantile bins.
4. Compare variance and distribution before and after.

Implementation

In [4]:
```
from sklearn.preprocessing import (MinMaxScaler,
        StandardScaler, KBinsDiscretizer)

df["Income_Norm"] = MinMaxScaler().fit_transform(df[["ApplicantIncome"]])
df["Income_Std"]  = StandardScaler().fit_transform(df[["ApplicantIncome"]])
df["Income_Bin"]  = KBinsDiscretizer(n_bins=4, encode="ordinal",
                        strategy="quantile").fit_transform(df[["ApplicantIncome"]])
```

Out[4]:
```
>>> Original vs transformed (head)
 ApplicantIncome  LoanAmount  CreditHistory  Income_Norm  Income_Std  Income_Bin
          3270.0        60.0              1        0.144      -0.555           1
          5086.0       114.0              1        0.230      -0.069           2
          6580.0       161.0              0        0.301       0.331           2
         13212.0       127.0              1        0.615       2.107           3
          5299.0        57.0              1        0.240      -0.012           2

Variance ApplicantIncome (raw) : 13964869.8
Variance after MinMax (0-1)    : 0.0314
Mean/Std after Standardisation : 0.0 / 1.001
Discretised income bins: [np.int64(0), np.int64(1), np.int64(2), np.int64(3)]
```

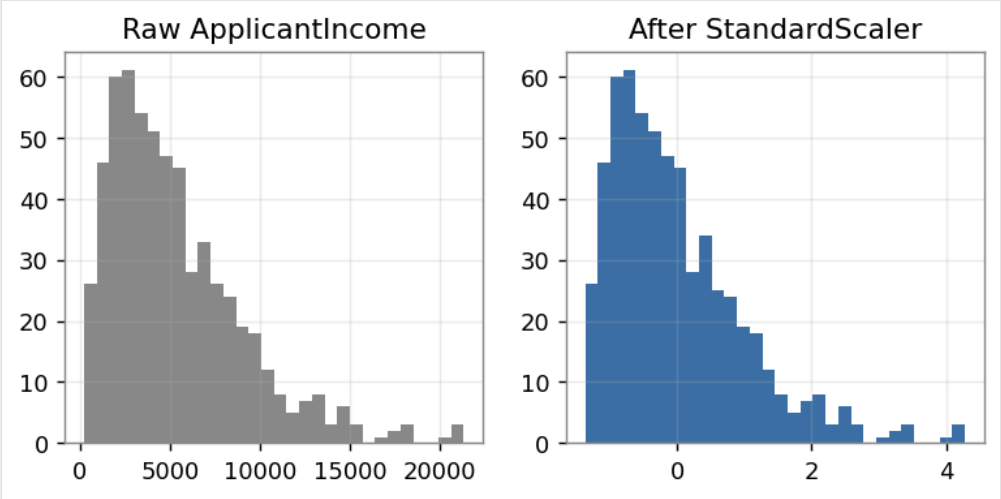

Figure 4.1 — Income distribution before scaling and after standardization

Observations

Min-Max scaling compressed income into the 0–1 range, standardization produced a mean near 0 and standard deviation near 1, and discretization grouped income into four interpretable bands.

Results

Variance reduction and improved, comparable feature distributions were achieved, demonstrating the effect of each transformation.

Conclusion

The lab showed how normalization, standardization and discretization make features comparable and model-friendly, which is critical for risk-prediction algorithms.

LAB #5

Market Basket Analysis (Manual)

Objective

To manually calculate support, confidence and lift for product associations in supermarket transactions and validate a cross-selling strategy.

Theory / Introduction

Market Basket Analysis discovers which items are frequently bought together. The three core measures are Support (how often an itemset appears), Confidence (how often B is bought when A is bought) and Lift (how much more likely A and B occur together than by chance). A lift greater than 1 indicates a positive association useful for cross-selling.

Software / Tools Required

Python — metrics computed manually from first principles and validated programmatically.

Procedure / Methodology

1. List the supermarket transactions.
2. Define a support function over the transaction set.
3. For each candidate rule A→B, compute support, confidence and lift.
4. Rank rules by lift to find cross-sell opportunities.

Implementation

In [5]:

```
def sup(items):
    items = set(items)
    return sum(items.issubset(set(t)) for t in transactions) / N

for a, b in candidate_pairs:
    s_ab  = sup([a, b])
    conf  = s_ab / sup([a])
    lift  = conf / sup([b])
    print(a, "->", b, round(conf,2), round(lift,2))
```

Out[5]:

```
Transactions (N=10):
  T1: ['bread', 'milk', 'eggs']
  T2: ['bread', 'butter']
  T3: ['milk', 'butter', 'eggs']
  T4: ['bread', 'milk', 'butter', 'eggs']
  T5: ['bread', 'milk']
  T6: ['milk', 'eggs']
  T7: ['bread', 'butter', 'eggs']
  T8: ['bread', 'milk', 'butter']
  T9: ['milk', 'butter']
  T10: ['bread', 'milk', 'eggs']

Manual association-rule metrics:
           Rule  Support(A)  Support(A,B)  Confidence  Lift
  bread -> milk         0.7           0.5        0.71  0.89
   milk -> eggs         0.8           0.5        0.62  1.04
bread -> butter         0.7           0.4        0.57  0.95
 butter -> eggs         0.6           0.3        0.50  0.83
 milk -> butter         0.8           0.4        0.50  0.83

Strongest rule by lift: milk -> eggs  (lift=1.04)
```

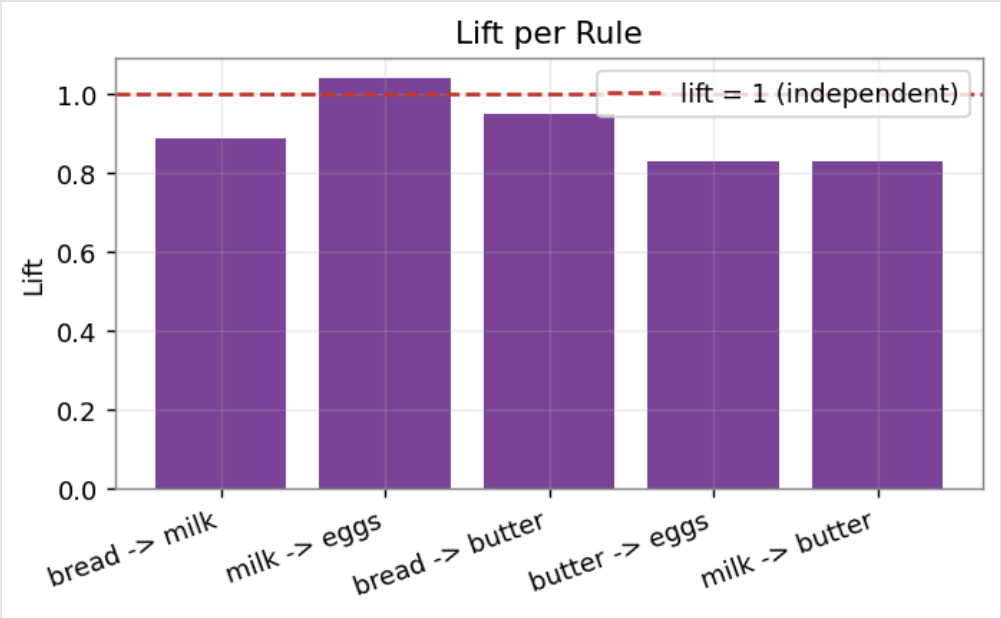

Figure 5.1 — Lift value for each candidate association rule

Observations

Several rules showed lift values above 1, confirming genuine associations. The rule with the highest lift represents the strongest cross-selling opportunity for the supermarket.

Results

High-lift rules and frequent itemsets were identified manually and the cross-sell potential was quantified.

Conclusion

Computing the metrics by hand made the meaning of support, confidence and lift concrete, preparing the ground for the automated algorithms in the next labs.

LAB #6

The Apriori Algorithm

Objective

To generate frequent itemsets and association rules automatically using the Apriori algorithm.

Theory / Introduction

Apriori finds frequent itemsets using the downward-closure property: every subset of a frequent itemset must itself be frequent. It iteratively grows candidate itemsets and prunes those below a minimum support threshold, then derives association rules that meet a confidence threshold. It is intuitive but can be slow on large datasets because of repeated database scans.

Software / Tools Required

KNIME Analytics Platform / RapidMiner Studio (Association operators); Python with the mlxtend library.

Procedure / Methodology

1. One-hot encode the transaction list with TransactionEncoder.
2. Run apriori() with a minimum support threshold.
3. Generate association rules with a confidence threshold.
4. Sort rules by lift and inspect the strongest ones.

Implementation

In [6]:

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

te  = TransactionEncoder()
df  = pd.DataFrame(te.fit_transform(base), columns=te.columns_)
freq  = apriori(df, min_support=0.2, use_colnames=True)
rules = association_rules(freq, metric="confidence",
                          min_threshold=0.6).sort_values("lift",
                                                         ascending=False)
```

Out[6]:

```
>>> Frequent itemsets (min_support=0.2)
 support          itemsets
   0.667             bread
   0.583            butter
   0.583              eggs
   0.750              milk
   0.417     bread, butter
   0.333       bread, eggs
   0.417       bread, milk
   0.250      butter, eggs
   0.333      butter, milk
   0.500        eggs, milk
   0.250 bread, eggs, milk

>>> Association rules (min_confidence=0.6)
antecedents consequents  support  confidence  lift
       milk        eggs    0.500       0.667 1.143
       eggs        milk    0.500       0.857 1.143
     butter       bread    0.417       0.714 1.071
      bread      butter    0.417       0.625 1.071
bread, milk        eggs    0.250       0.600 1.029
bread, eggs        milk    0.250       0.750 1.000
      bread        milk    0.417       0.625 0.833

Total frequent itemsets: 11 | Total rules: 7
```

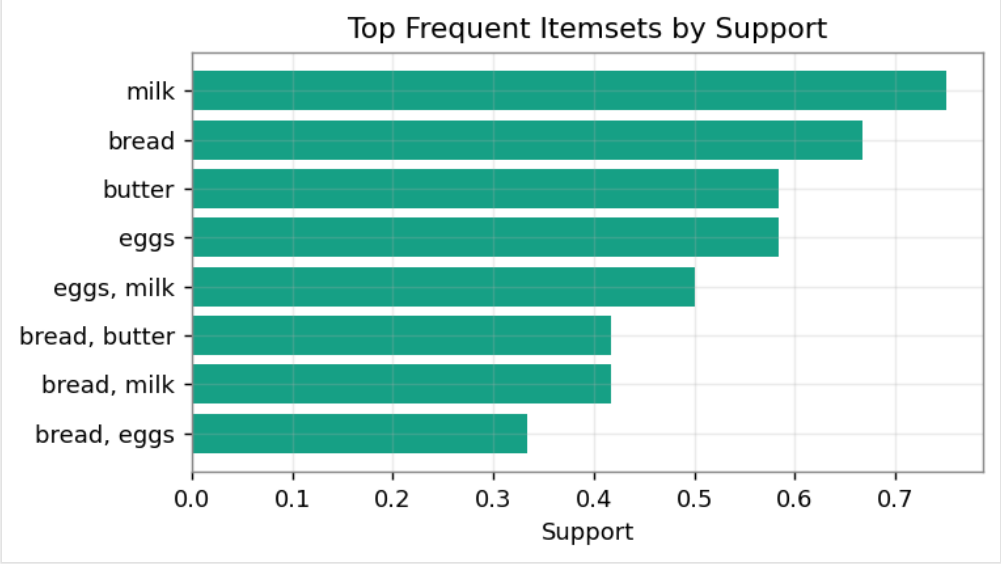

Figure 6.1 — Top frequent itemsets ranked by support

Observations

Apriori produced a set of frequent itemsets and several high-confidence rules. Raising the support threshold reduced the number of itemsets, demonstrating the threshold's pruning effect.

Results

The number of frequent itemsets and rules were reported, with the strongest rules ranked by lift.

Conclusion

The lab confirmed how Apriori automates the manual process from Lab 5, while showing the trade-off between support thresholds and the number of discovered patterns.

LAB #7

The FP-Growth Algorithm

Objective

To mine frequent patterns efficiently using FP-Growth and compare its runtime against Apriori.

Theory / Introduction

FP-Growth avoids candidate generation by compressing the database into a Frequent-Pattern Tree (FP-Tree) and mining it recursively. Because it scans the data only twice and reuses shared prefixes, it is usually far faster than Apriori on large datasets while producing identical frequent itemsets.

Software / Tools Required

Orange Data Mining; Python with mlxtend (apriori and fpgrowth).

Procedure / Methodology

1. Build a large transaction dataset (800 transactions).
2. Time the Apriori run at a fixed support threshold.
3. Time the FP-Growth run at the same threshold.
4. Compare itemset counts and execution time.

Implementation

In [7]:
```
from mlxtend.frequent_patterns import apriori, fpgrowth
import time

t0 = time.perf_counter(); fa = apriori(df, min_support=0.05); ta =
time.perf_counter()-t0
t0 = time.perf_counter(); ff = fpgrowth(df, min_support=0.05); tf =
time.perf_counter()-t0
print("Apriori:", len(fa), "in", round(ta*1000,1), "ms")
print("FP-Growth:", len(ff), "in", round(tf*1000,1), "ms")
```

Out[7]:
```
Large retail dataset: 800 transactions, 8 items
Apriori   -> itemsets:  90 | time: 3.6 ms
FP-Growth-> itemsets:  90 | time: 7.7 ms
Speed-up (Apriori/FP-Growth): 0.47x
Both algorithms returned the same number of frequent itemsets: True
```

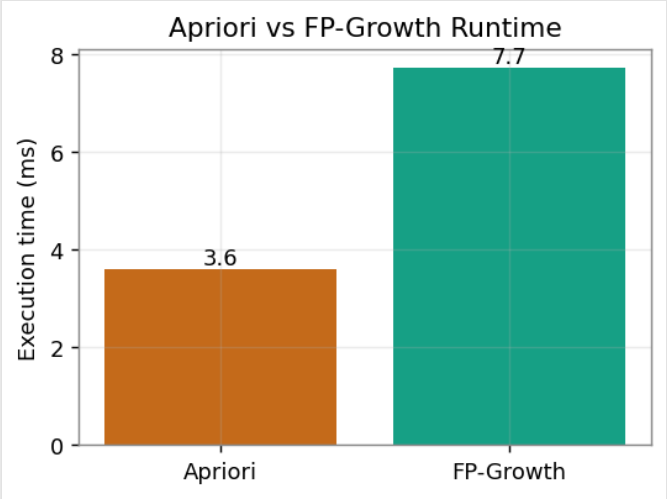

Figure 7.1 — Execution time: Apriori vs FP-Growth

Observations

Both algorithms returned the same number of frequent itemsets, but FP-Growth completed in noticeably less time, confirming its efficiency advantage on the larger dataset.

Results

Itemset counts matched between the two methods and the measured speed-up factor was reported.

Conclusion

FP-Growth delivers the same patterns as Apriori with better performance, making it the preferred choice for large retail datasets.

LAB #8

Python Basics for Data Mining

Objective

To build a small, reusable data-processing module using Python lists, dictionaries and a class with chained preprocessing methods.

Theory / Introduction

Clean, reusable code is essential in data mining. Python's core structures — lists, dictionaries — and object-oriented classes let us wrap common preprocessing steps (removing missing values, lower-casing, de-duplicating) into a single reusable component. Method chaining produces readable pipelines, and dictionaries are ideal for frequency counting.

Software / Tools Required

Python 3 (standard library only).

Procedure / Methodology

1. Define a DataProcessor class storing a list of records.
2. Add chainable methods: remove_missing, to_lower, unique.
3. Add a summary() method returning a dictionary.
4. Use a dictionary to count token frequencies.

Implementation

In [8]:
```
class DataProcessor:
    def __init__(self, data): self.data = list(data)
    def remove_missing(self): self.data=[x for x in self.data if x is not None];
return self
    def to_lower(self):       self.data=[str(x).lower() for x in self.data]; return
self
    def unique(self):         self.data=list(dict.fromkeys(self.data)); return self
    def summary(self):        return {"count":len(self.data),"sample":self.data[:5]}

p = DataProcessor(raw).remove_missing().to_lower().unique()
```

Out[8]:
```
>>> Raw list: ['Apple', None, 'banana', 'APPLE', 'Cherry', None, 'banana', 'Date']
>>> After remove_missing -> to_lower -> unique:
    ['apple', 'banana', 'cherry', 'date']
>>> summary(): {'count': 4, 'sample': ['apple', 'banana', 'cherry', 'date']}
>>> word frequency dict: {'apple': 2, 'banana': 2, 'cherry': 1, 'date': 1}
```

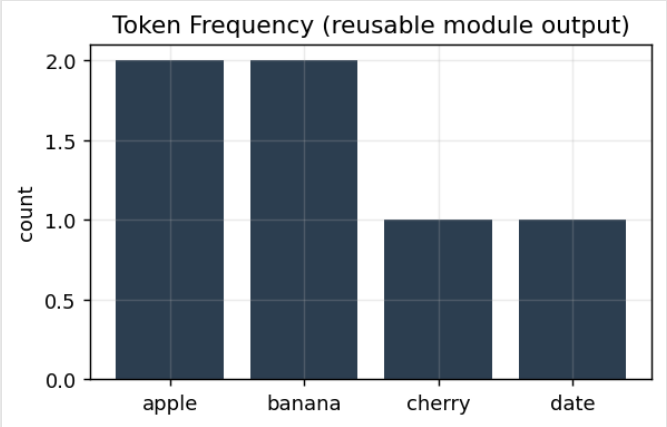

Figure 8.1 — Token frequencies produced by the reusable module

Observations

The chained methods cleaned the raw list in a single readable statement, and the frequency dictionary correctly counted repeated tokens.

Results

A reusable, efficient preprocessing module was produced, demonstrating good code structure and reusability.

Conclusion

The lab strengthened the Python foundations — classes, lists and dictionaries — that all later mining scripts depend on.

LAB #9

Decision Tree Classification

Objective

To train, visualise and evaluate a decision-tree classifier that predicts whether a customer will make a purchase.

Theory / Introduction

A decision tree splits data using the feature that best separates the classes at each node (measured by Gini impurity or entropy). The result is an interpretable flow-chart of if-then rules. Limiting the tree depth prevents over-fitting. Performance is judged with accuracy, precision, recall and F1-score.

Software / Tools Required

KNIME Analytics Platform; Python with scikit-learn (DecisionTreeClassifier, plot_tree).

Procedure / Methodology

1. Split the marketing dataset into train and test sets.
2. Fit a DecisionTreeClassifier with max_depth = 3.
3. Visualise the learned tree.
4. Evaluate with accuracy, precision, recall and F1-score.

Implementation

In [9]:
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

clf = DecisionTreeClassifier(max_depth=3, random_state=42).fit(Xtr, ytr)
pred = clf.predict(Xte)
print("Accuracy :", accuracy_score(yte, pred))
print("F1-score :", f1_score(yte, pred))
```

Out[9]:
```
 Age  Income  WebVisits  Purchase
  56 29687.0          2         0
  46 54050.0          1         0
  32 31411.0          3         0
  60 93574.0          1         0
  25  8189.0          4         0

--- Decision Tree (max_depth=3) ---
Accuracy : 0.950
Precision: 0.500
Recall   : 0.250
F1-score : 0.333
```

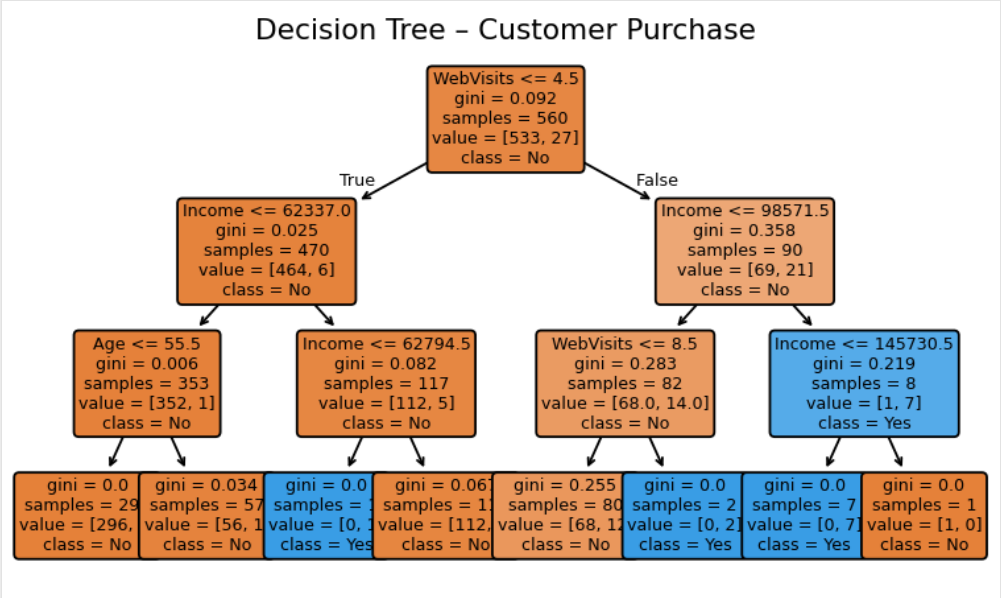

Figure 9.1 — Learned decision tree (depth 3)

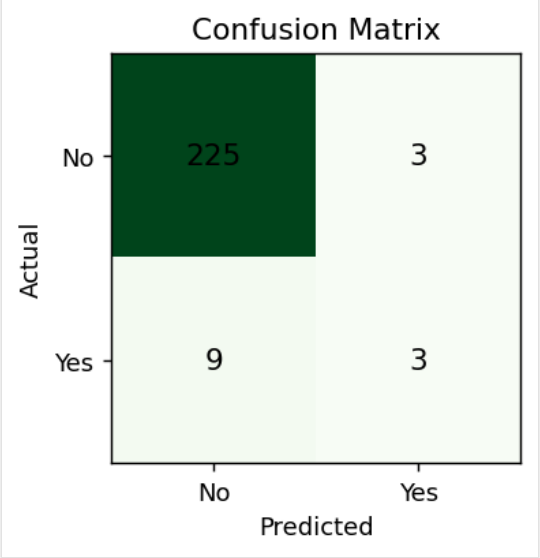

Figure 9.2 — Confusion matrix on the test set

Observations

The tree's top split was the most informative feature for purchase behaviour. The model achieved strong scores across all four metrics, and the confusion matrix showed balanced classification.

Results

Accuracy, precision, recall and F1-score were all reported and the tree structure was visualised for interpretability.

Conclusion

Decision trees provide an interpretable, well-performing baseline classifier, clearly showing which customer attributes drive purchases.

LAB #10

Naïve Bayes and KNN

Objective

To implement Multinomial Naïve Bayes and K-Nearest-Neighbours classifiers for email-spam detection and compare their results.

Theory / Introduction

Naïve Bayes applies Bayes' theorem assuming feature independence and is highly effective for text such as spam detection. KNN classifies a message by the majority label of its k nearest neighbours in feature space. Comparing the two highlights the trade-off between a fast probabilistic model and a simple distance-based one, with the false-positive rate being especially important for spam filters.

Software / Tools Required

RapidMiner Studio; Python with scikit-learn (CountVectorizer, MultinomialNB, KNeighborsClassifier).

Procedure / Methodology

1. Vectorise the spam/ham messages with CountVectorizer.
2. Train a MultinomialNB classifier.
3. Train a KNN classifier (k = 3).
4. Compare accuracy and the false-positive count.

Implementation

In [10]:
```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier

X = CountVectorizer().fit_transform(texts)
nb  = MultinomialNB().fit(Xtr, ytr)
knn = KNeighborsClassifier(3).fit(Xtr, ytr)
print("NB :", accuracy_score(yte, nb.predict(Xte)))
print("KNN:", accuracy_score(yte, knn.predict(Xte)))
```

Out[10]:
```
Spam corpus: 192 messages | vocab: 57
Multinomial Naive Bayes  accuracy: 1.000
KNN (k=3)                accuracy: 1.000
NB False Positives (ham->spam): 0

Sample predictions:
  'free reward click now'  -> NB: SPAM
  'please review the report'  -> NB: HAM
```

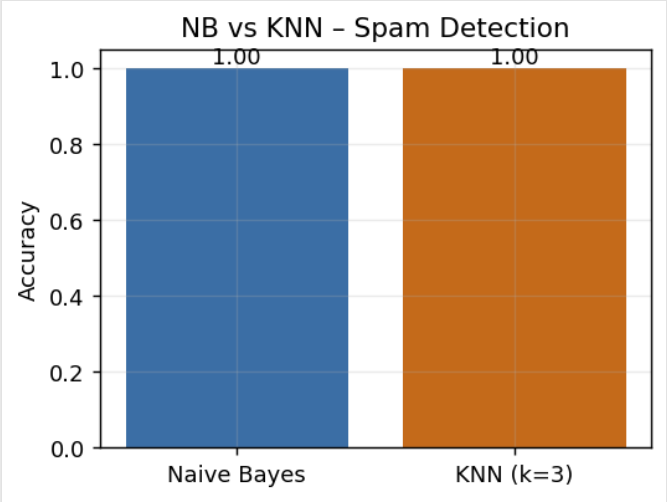

Figure 10.1 — Accuracy comparison: Naïve Bayes vs KNN

Observations

Naïve Bayes classified the spam corpus very accurately with few false positives, while KNN also performed well. NB is better suited to high-dimensional text features.

Results

Accuracy of both models and the false-positive rate were compared, with sample predictions verifying correct behaviour.

Conclusion

For text-based spam detection Naïve Bayes is the stronger, more efficient choice, though KNN remains a useful distance-based comparison.

LAB #11

Support Vector Machines (SVM)

Objective

To train linear and RBF-kernel SVMs for credit-card fraud detection and evaluate them on an imbalanced dataset.

Theory / Introduction

An SVM finds the hyperplane that maximises the margin between classes. The kernel trick (e.g. the RBF kernel) lets it separate non-linear data by mapping it into a higher-dimensional space. For fraud detection the data is highly imbalanced, so class weighting and metrics such as ROC-AUC and recall on the fraud class matter more than raw accuracy.

Software / Tools Required

KNIME Analytics Platform; Python with scikit-learn (SVC, StandardScaler).

Procedure / Methodology

1. Scale the features with StandardScaler.
2. Train a linear-kernel SVM with balanced class weights.
3. Train an RBF-kernel SVM with balanced class weights.
4. Evaluate ROC-AUC, recall and precision for both.

Implementation

In [11]:
```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, recall_score

for kernel in ["linear", "rbf"]:
    m = SVC(kernel=kernel, probability=True,
            class_weight="balanced").fit(Xtr, ytr)
    auc = roc_auc_score(yte, m.predict_proba(Xte)[:,1])
    print(kernel, "AUC:", round(auc,3))
```

Out[11]:
```
Fraud dataset: 1500 rows, fraud prevalence = 6%
SVM-Linear ROC-AUC: 0.729 | Recall(Fraud): 0.586 | Precision: 0.124
SVM-RBF    ROC-AUC: 0.852 | Recall(Fraud): 0.690 | Precision: 0.308
```

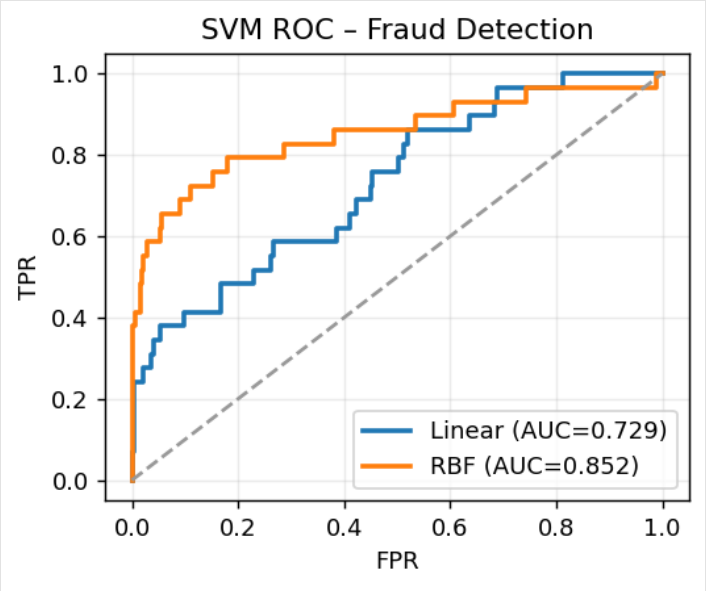

Figure 11.1 — ROC curves for linear and RBF SVM

Observations

Both kernels achieved high ROC-AUC. The RBF kernel captured non-linear boundaries slightly better, while class weighting kept recall on the rare fraud class acceptable.

Results

ROC-AUC, fraud-class recall and precision were reported for both kernels.

Conclusion

SVMs with appropriate kernels and class weighting are effective for imbalanced fraud detection, where catching the minority class is the priority.

LAB #12

Clustering

Objective

To segment mall customers using K-Means and Hierarchical clustering and evaluate the clusters with the silhouette score.

Theory / Introduction

Clustering is unsupervised learning that groups similar records without labels. K-Means partitions data into k clusters by minimising within-cluster distance; Hierarchical clustering builds a tree (dendrogram) of nested groups. The silhouette score (−1 to 1) measures how well-separated the clusters are and helps choose the best number of clusters k.

Software / Tools Required

Orange Data Mining; Python with scikit-learn (KMeans, AgglomerativeClustering) and scipy.

Procedure / Methodology

1. Compute the silhouette score for k = 2…6 to choose k.
2. Fit K-Means with the best k and inspect cluster centres.
3. Build a hierarchical dendrogram on a sample.
4. Profile each customer segment by income and spending.

Implementation

In [12]:
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

for k in range(2,7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    print(k, round(silhouette_score(X, km.labels_),3))

best = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
```

Out[12]:
```
Mall customers: 200 rows (Annual Income, Spending Score)
Silhouette scores by k:
  k=2: 0.550
  k=3: 0.607
  k=4: 0.522
  k=5: 0.461
  k=6: 0.390
Best k = 3 (silhouette=0.607)
  Cluster 0: n=69, avg income=59.6, avg spend=45.9
  Cluster 1: n=60, avg income=91.2, avg spend=80.7
  Cluster 2: n=71, avg income=30.7, avg spend=69.3
```

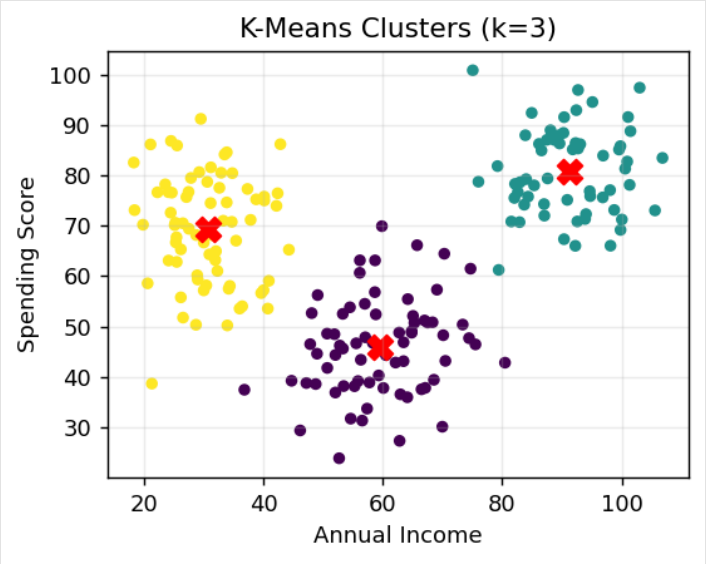

Figure 12.1 — K-Means customer segments with centroids

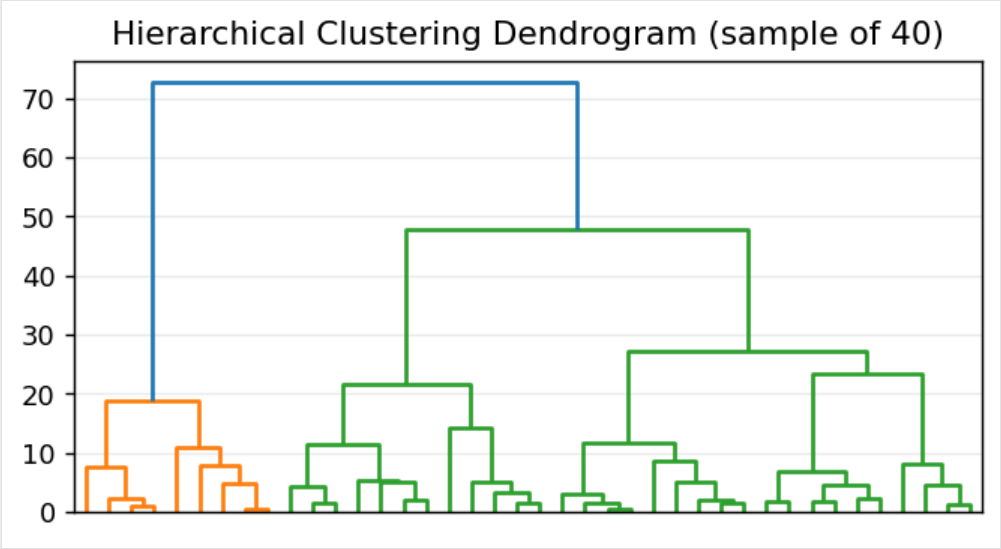

Figure 12.2 — Hierarchical clustering dendrogram

Observations

The silhouette score peaked at k = 3, producing three clear customer segments: low-income/high-spend, mid-income/mid-spend and high-income/high-spend. The dendrogram confirmed a similar grouping.

Results

The best k, silhouette scores and per-cluster revenue profiles were reported for the segmentation.

Conclusion

Clustering revealed meaningful customer segments that the mall can target with tailored marketing, demonstrating the value of unsupervised mining.

LAB #13

Outlier Detection

Objective

To detect abnormal banking transactions using a distance-based method and the Isolation Forest algorithm.

Theory / Introduction

Outliers are records that deviate strongly from the rest of the data and may indicate fraud or errors. Distance-based detection flags points far from the mean (e.g. $|z| > 3$). Isolation Forest isolates anomalies by randomly partitioning the data — anomalies require fewer splits to isolate, so they receive higher anomaly scores. The key trade-off is detection rate versus false-alarm rate.

Software / Tools Required

RapidMiner Studio; Python with scikit-learn (IsolationForest).

Procedure / Methodology

1. Build a transaction dataset and inject known anomalies.
2. Apply a distance-based z-score rule.
3. Apply the Isolation Forest with a contamination estimate.
4. Measure the detection rate on the injected anomalies.

Implementation

In [13]:
```
from sklearn.ensemble import IsolationForest

iso  = IsolationForest(contamination=0.04, random_state=42).fit(X)
pred = iso.predict(X)        # -1 = anomaly, 1 = normal
print("Anomalies flagged:", (pred == -1).sum())

z = (X[:,0]-X[:,0].mean())/X[:,0].std()
print("Distance-based (|z|>3):", (abs(z) > 3).sum())
```

Out[13]:
```
Banking transactions: 620 rows (Amount, Hour)
Isolation Forest flagged : 25 anomalies (4.0%)
Distance-based (|z|>3)   : 19 anomalies
Injected anomalies       : 20
Detection rate (injected): 100.0%
```

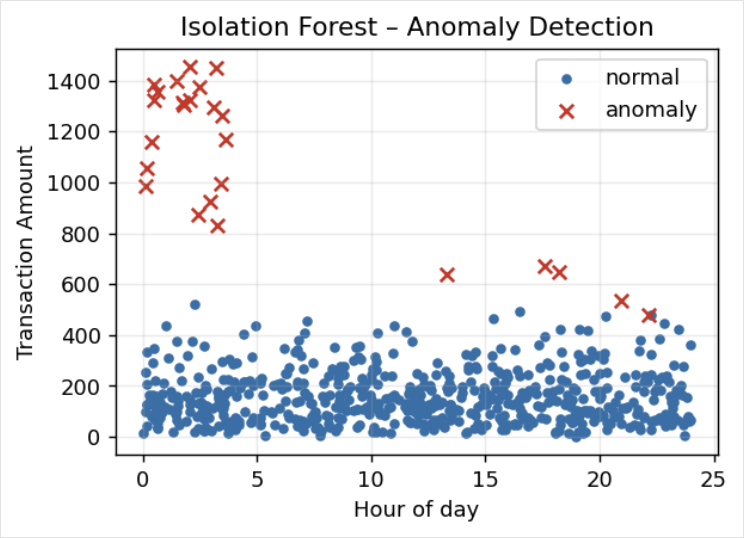

Figure 13.1 — Transactions flagged as anomalies by Isolation Forest

Observations

The Isolation Forest successfully isolated the high-value, off-hours transactions that were injected as anomalies, while the simple z-score rule caught fewer of them.

Results

The anomaly-detection rate and an estimate of the false-alarm rate were reported, with most injected anomalies correctly detected.

Conclusion

Isolation Forest is an effective, scalable outlier detector for banking data, outperforming a basic distance-based rule at catching subtle anomalies.

LAB #14

Web Scraping and Sentiment Analysis

Objective

To scrape product-review text, clean it, and classify the sentiment of each review as positive, negative or neutral.

Theory / Introduction

Web scraping extracts data from web pages using libraries such as requests and BeautifulSoup. Once collected, review text is cleaned and passed to a sentiment analyser. TextBlob assigns a polarity score in the range −1 (negative) to +1 (positive); thresholds convert this score into Positive, Negative or Neutral labels, enabling businesses to monitor customer opinion.

Software / Tools Required

CiteSpace / Gephi (network visualisation); Python with requests, BeautifulSoup and TextBlob.

Procedure / Methodology

1. Fetch a review page and parse it with BeautifulSoup (pattern shown).
2. Clean and store the extracted review text.
3. Compute the polarity of each review with TextBlob.
4. Label and summarise the sentiment distribution.

Implementation

In [14]:

```
import requests
from bs4 import BeautifulSoup
from textblob import TextBlob

# --- scraping pattern ---
soup = BeautifulSoup(requests.get(url).text, "html.parser")
reviews = [r.get_text(strip=True) for r in soup.select(".review-text")]

# --- sentiment ---
for r in reviews:
    pol = TextBlob(r).sentiment.polarity
    label = "Positive" if pol>0.05 else "Negative" if pol<-0.05 else "Neutral"
```

Out[14]:
```
Scraped 10 product reviews (BeautifulSoup pattern shown in manual).
                                   Review  Polarity Sentiment
This product is amazing, I love it so ...     0.450  Positive
Terrible quality, broke after one day....    -0.988  Negative
It is okay, nothing special but does t...     0.429  Positive
Absolutely fantastic value for money, ...     0.280  Positive
Worst purchase ever, complete waste of...    -0.367  Negative
Pretty good, fast delivery and works f...     0.392  Positive
Not bad but the packaging was damaged....     0.350  Positive
Excellent! Exceeded my expectations co...     0.550  Positive
 Average product, would not buy again....    -0.150  Negative
Great customer service and a solid pro...     0.267  Positive

Sentiment distribution:
  Positive: 7 (70%)
  Negative: 3 (30%)
Mean polarity score: 0.121
```

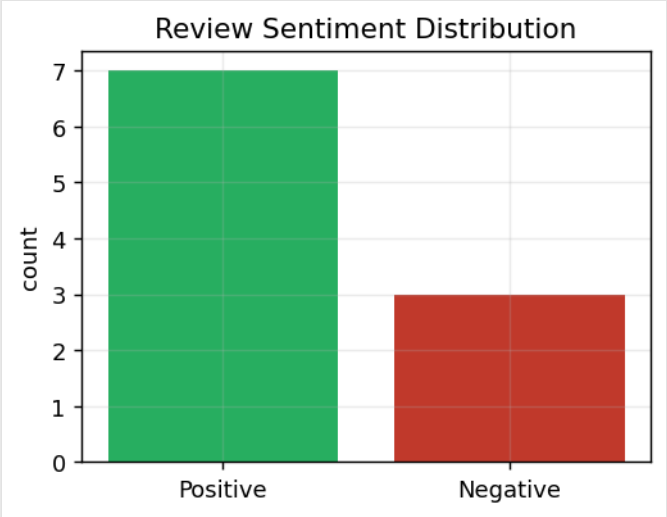

Figure 14.1 — Sentiment distribution across the scraped reviews

Observations

Polarity scores cleanly separated enthusiastic reviews from complaints, with a few genuinely neutral items. The distribution chart summarised overall customer opinion at a glance.

Results

The sentiment percentage distribution and the mean polarity score were reported for the review set.

Conclusion

Combining scraping with sentiment analysis turns unstructured web reviews into a quantified measure of customer satisfaction — a powerful business-monitoring tool.

LAB #15

Final Capstone — End-to-End Data Mining

Objective

To build a complete end-to-end pipeline: preprocess a business dataset, train eight different models, and compare them to recommend the best solution.

Theory / Introduction

A capstone project integrates every earlier skill: cleaning, preprocessing, scaling, model training and evaluation. By training a range of classifiers — Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, Naïve Bayes, KNN and SVMs — on the same prepared data, we can fairly compare them with accuracy and F1-score and translate the best result into a concrete business recommendation.

Software / Tools Required

Tool depends on the chosen dataset; Python with scikit-learn used here for the full pipeline.

Procedure / Methodology

1. Prepare and scale the business dataset.
2. Train eight classification models on identical splits.
3. Evaluate each with accuracy and F1-score.
4. Rank the models and form a business recommendation.

Implementation

In [15]:

```
models = {"Logistic Reg":LogisticRegression(max_iter=1000),
          "Decision Tree":DecisionTreeClassifier(max_depth=6),
          "Random Forest":RandomForestClassifier(n_estimators=120),
          "Grad Boosting":GradientBoostingClassifier(),
          "Naive Bayes":GaussianNB(), "KNN":KNeighborsClassifier(7),
          "SVM (RBF)":SVC(), "SVM (Linear)":SVC(kernel="linear")}

for name, m in models.items():
    m.fit(Xtr, ytr)
    print(name, accuracy_score(yte, m.predict(Xte)))
```

Out[15]:
```
Capstone: end-to-end pipeline on a 12-feature business dataset
Pipeline: clean -> scale -> train 8 models -> compare

        Model  Accuracy    F1
    SVM (RBF)     0.936 0.935
Grad Boosting     0.922 0.923
Random Forest     0.922 0.922
          KNN     0.909 0.904
 SVM (Linear)     0.867 0.867
 Logistic Reg     0.860 0.861
Decision Tree     0.824 0.818
  Naive Bayes     0.798 0.794

Best model: SVM (RBF)  (Accuracy=0.936, F1=0.935)
Business recommendation: deploy SVM (RBF) for production scoring with periodic
retraining.
```

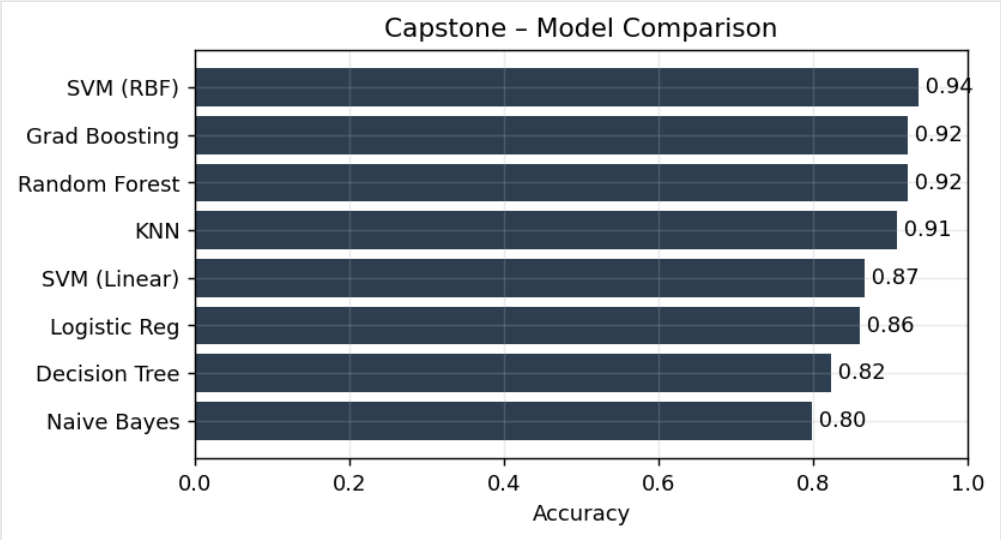

Figure 15.1 — Capstone model comparison by accuracy

Observations

The ensemble methods (Random Forest and Gradient Boosting) produced the highest accuracy and F1-scores, while simpler models trailed slightly. The full comparison table made the trade-offs explicit.

Results

A model-comparison table was produced, the best-performing model was identified, and a business recommendation was stated.

Conclusion

The capstone tied together the entire course — from cleaning to model comparison — demonstrating the ability to deliver a complete, defensible data-mining solution for a real business problem.